PROMOTION B2 — ÉVALUATION PRATIQUE

Programmation Orientée Objet (Java)

Durée de l'épreuve : 2 heures | **Langage imposé :** Java | **Point d'entrée :** `Main.java`

Règlement de l'évaluation et modalités de restitution :

- **Langage imposé :** L'intégralité des exercices doit impérativement être codée en **Java**.
- **Accès à Internet :** L'accès à Internet est ouvert (documentation officielle, forums, tutoriels, ressources de cours).En revanche, **l'utilisation de toute Intelligence Artificielle générative** (ChatGPT, Claude, GitHub Copilot, Gemini, etc.) est **strictement interdite** et sera sanctionnée comme une fraude.
- **Exécution dans `Main.java` :** Votre projet doit contenir un fichier racine `Main.java` possédant la méthode `public static void main(String[] args)`. Il ne suffit pas de déclarer vos classes : vous devez impérativement suivre les consignes d'exécution de chaque exercice dans `main()` pour instancier vos objets et afficher les retours de vos méthodes dans la console.
- **Rendu sur GitHub :** Votre travail doit être hébergé et poussé sur un dépôt GitHub nommé exactement `eval_poo_nom_prenom` (en remplaçant par votre nom et votre prénom). La version finale de votre code doit être validée par un commit portant exactement le message : `Terminé`.

Exercice 1 — Encapsulation : Gestion de stock d'un article

Vous devez modéliser une classe représentant un article en stock dans un entrepôt. Pour garantir la cohérence de l'inventaire, la quantité disponible ne doit jamais être accessible ou modifiée directement depuis l'extérieur de la classe.

1. Créer une classe nommée `ArticleStock`.
2. Définir les propriétés pour le nom de l'article (`nom`) et sa quantité en stock (`quantite`). Assurez-vous de faire en sorte que ces propriétés soient inaccessibles à l'extérieur de la classe.
3. Implémenter le constructeur permettant d'initialiser l'article avec son nom et une quantité de départ (par défaut à `0` si elle n'est pas spécifiée).
4. Implémenter les méthodes Getter (`getNom()` et `getQuantite()`) pour permettre de consulter le nom et la quantité disponible.
5. Implémenter deux méthodes publiques pour gérer les mouvements de stock :
   - `approvisionner(nb)` : ajoute la quantité fournie au stock (uniquement si elle est strictement positive).
   - `expedier(nb)` : soustrait la quantité fournie du stock et retourne un message indiquant le résultat de l'opération.

6. Ajouter une validation dans `expedier(nb)` : le retrait ne doit être possible que si la quantité demandée est inférieure ou égale au stock actuel. Si l'expédition réussit, la méthode retourne `"Expédition de [nb] unités validée."`. Si elle échoue, le stock reste inchangé et la méthode retourne `"Erreur : stock insuffisant pour expédier [nb] unités."`.

7. **Dans `Main.java` (méthode `main()`) :** Instancier un objet `ArticleStock` nommé `"Clavier mécanique"` sans préciser de quantité initiale. Appeler `approvisionner(15)`, puis afficher dans la console ce que la méthode `getQuantite()` nous retourne. Appeler ensuite `expedier(6)` puis `expedier(12)` en affichant dans la console ce que chacune de ces deux méthodes nous retourne, et enfin afficher une dernière fois le retour de `getQuantite()`.

Exercice 2 — Héritage et Polymorphisme : Appareils connectés

Vous modélisez un système domotique gérant différents appareils connectés. Tous les appareils sont installés dans une pièce de la maison (ex: Salon, Cuisine) et émettent un rapport d'état, mais chaque sous-catégorie d'appareil a un comportement légèrement différent.

1. Créer une classe parente nommée `AppareilConnecte` contenant :
   - Une propriété `piece` (représentant le nom de la pièce de la maison où est installé l'appareil).
   - Un constructeur prenant en paramètre le nom de cette pièce afin d'initialiser la propriété `piece`.
   - Une méthode `diagnostiquer()` qui retourne : `"L'appareil situé dans [piece] fonctionne normalement."`
2. Créer une classe fille nommée `Thermostat` qui hérite de `AppareilConnecte` :
   - Redéfinir la méthode `diagnostiquer()` pour retourner : `"Le thermostat de [piece] régule la température à 20°C."`
   - Ajouter une méthode spécifique `activerModeEco()` qui retourne : `"Le thermostat de [piece] passe en mode économique."`
3. Créer une autre classe fille nommée `CameraSecurite` qui hérite de `AppareilConnecte` :
   - Redéfinir la méthode `diagnostiquer()` pour retourner : `"La caméra de [piece] filme en haute définition."`

4. **Dans `Main.java` (méthode `main()`) :** Instancier un `AppareilConnecte` dans `"Couloir"`, un `Thermostat` dans `"Salon"` et une `CameraSecurite` dans `"Entrée"`. Afficher dans la console ce que la méthode `diagnostiquer()` nous retourne pour chacun de ces trois objets. Afficher également dans la console ce que la méthode `activerModeEco()` nous retourne sur le thermostat.

Exercice 3 — Abstraction : Système de réservation d'hébergements

Vous modélisez un système de calcul de prix pour des réservations d'hébergements. Tous les hébergements ont des caractéristiques communes, mais la formule de calcul du tarif d'un séjour diffère selon le type de bien.

1. Créer une classe abstraite nommée `Hebergement` qui ne peut pas être instanciée directement :
   - Ajouter les propriétés protégées `reference` et `tarifJournalier`.
   - Créer un constructeur qui initialise `reference` et `tarifJournalier`.
   - Déclarer une méthode abstraite : `calculerPrixSejour(nbNuits)`.
   - Créer une méthode concrète `decrire()` qui retourne : `"Hébergement [reference] - Tarif de base : [tarifJournalier] €/nuit"`.
2. Créer une classe concrète `ChambreHotel` qui hérite de `Hebergement` :
   - Ajouter une propriété `supplementPetitDejeuner` à la chambre d'hôtel.
   - Implémenter `calculerPrixSejour(nbNuits)` : le prix du séjour correspond au tarif journalier majoré du supplément petit-déjeuner, le tout multiplié par le nombre de nuits.
3. Créer une classe concrète `AppartementLocatif` qui hérite de `Hebergement` :
   - Ajouter une propriété `fraisMenageFixes` à l'appartement.
   - Implémenter `calculerPrixSejour(nbNuits)` : le prix du séjour correspond au tarif journalier multiplié par le nombre de nuits, auquel on ajoute une seule fois les frais de ménage fixes.

---

4. **Dans `Main.java` (méthode `main()`) :** Instancier une `ChambreHotel` (référence `"CH-101"`, tarif `85.0`, petit-déjeuner `12.5`) et un `AppartementLocatif` (référence `"APT-4B"`, tarif `110.0`, frais de ménage `50.0`). Pour chacun des deux objets, afficher dans la console ce que la méthode `decrire()` nous retourne, puis afficher ce que la méthode `calculerPrixSejour(4)` nous retourne.

Exercice 4 — Relation entre classes : Smartphone et Batterie interne

Modéliser la relation entre une classe `Smartphone` et une classe `BatterieInterne`. Le smartphone ne peut pas exister sans sa batterie interne, et cette batterie n'a de sens que dans ce smartphone spécifique.

1. Expliquer brièvement en commentaire au-dessus de la classe `Smartphone` comment traduire dans votre code le fait que la batterie appartienne exclusivement à ce smartphone et ne puisse pas exister indépendamment de lui.
2. Créer la classe `BatterieInterne` avec une propriété `capaciteMah` et une méthode `getCapaciteMah()` retournant cette valeur.
3. Créer la classe `Smartphone` avec :
   - Une propriété `modele` et une propriété `batterie`.
   - Un constructeur qui crée et initialise directement son objet `BatterieInterne`.
   - Une méthode `obtenirFicheTechnique()` qui retourne : `"Smartphone [modele] équipé d'une batterie scellée de [capaciteMah] mAh."`

---

4. **Dans `Main.java` (méthode `main()`) :** Instancier un objet `Smartphone` (modèle `"Pixel Pro"`, capacité `5000` mAh) et afficher dans la console ce que la méthode `obtenirFicheTechnique()` nous retourne.

Exercice 5 — Design Pattern Singleton : Gestionnaire de journalisation

Vous devez créer une classe `JournalEvenements` qui ne doit avoir qu'une seule et unique instance dans toute l'application. Elle servira à centraliser les paramètres globaux de journalisation (logs).

1. Créer la classe `JournalEvenements`.
2. Définir une propriété statique pour stocker l'unique instance de la classe.
3. Rendre le constructeur privé (ou protégé) pour empêcher l'instanciation directe avec `new`.
4. Créer une méthode statique publique `getInstance()` qui est le seul point d'accès pour obtenir l'instance. Cette méthode doit :
   - Vérifier si l'instance existe déjà.
   - Si non, la créer et la stocker.
   - Retourner l'instance.
5. Ajouter une propriété `niveauAlerte` (initialisée à `"INFO"` par défaut) ainsi que les méthodes `getNiveauAlerte()` et `setNiveauAlerte(niveau)`.

---

6. **Dans `Main.java` (méthode `main()`) :** Récupérer l'instance dans une variable `journal1` et modifier le niveau d'alerte à `"CRITIQUE"` avec `setNiveauAlerte()`. Récupérer ensuite l'instance dans une seconde variable `journal2`. Afficher dans la console ce que `journal2.getNiveauAlerte()` nous retourne, puis afficher le résultat de `(journal1 == journal2)`.